SENSEX-NIFTY Algo Trading

Code Guide - what every script does and how to run it

Generated 2026-06-25 | P&L basis: Rs20/order = Rs80/round-trip (4 orders) | 13 active scripts in code/ , 23 archived in code/_archive/

0. Setup you need every time

Python (venv): always use the project venv - the base python is broken.

```
c:\Users\ADITYA\Desktop\Algo_trading\log_tradingVenv\Scripts\python.exe
```

Run pattern: most scripts read/write data in feed_data/, so cd there first and call code via ..\code\

```
cd c:\Users\ADITYA\Desktop\Algo_trading\feed_data
..\log_tradingVenv\Scripts\python.exe ..\code\<script>.py
```

Daily token: Upstox tokens expire every day. Before any live feed, log in once to refresh upstox_token.json (opens a browser; paste back the https://127.0.0.1/?code=... URL).

```
..\log_tradingVenv\Scripts\python.exe ..\code\live_fetch.py
```

1. Run the live feed -> dated folder + dated CSVs (automatic)

The feed does this for you now. Just run tick_poller.py from feed_data/ - it creates a folder named with today's date and writes two dated CSVs into it: sensex_<date>.csv and nifty_<date>.csv. It ALSO keeps sensex_today.csv / nifty_today.csv updated so the rest of the pipeline keeps working. Run during market hours (09:15-15:30 IST); it auto-stops at 15:30.

Copy-paste (PowerShell):

```
cd c:\Users\ADITYA\Desktop\Algo_trading\feed_data
..\log_tradingVenv\Scripts\python.exe ..\code\tick_poller.py
```

Result (folder created automatically):

```
feed_data\2026_06_25\sensex_2026-06-25.csv
feed_data\2026_06_25\nifty_2026-06-25.csv
feed_data\sensex_today.csv    (live mirror, for paper_tracker / live_trades_today)
feed_data\nifty_today.csv
```

Each row = one second, with every field the Upstox full-quote feed returns.

Add --trade to also paper-trade the spread in the Upstox sandbox; --intraday for the per-second signal; --interval 2 to poll every 2s; --lots N / --max-loss Rs / --max-trades N for risk caps. To force explicit paths instead of the dated folder, pass --sensex-out X --nifty-out Y (manual mode).

2. Daily routine (3 steps)

1) Morning - refresh token: ..\code\live_fetch.py
2) Market hours - collect feed (+ optional paper trade): ..\code\tick_poller.py (see section 1)
3) After 15:30 close - log the carry-forward signal: ..\code\paper_tracker.py

3. ACTIVE scripts (code/)

tick_poller.py [ACTIVE]

Purpose: Live engine: polls Upstox every second and appends the full SENSEX & NIFTY quote (price, OHLC, volume/OI from the nearest future, full raw payload) to CSV. With --trade it paper-trades the spread in the Upstox sandbox. Auto-stops 15:30.

Run:

```
# default: auto dated folder + dated files (+ today-mirror):
..\log_tradingVenv\Scripts\python.exe ..\code\tick_poller.py
# manual paths instead of the dated folder: --sensex-out P --nifty-out P
# flags: --interval N  --no-save  --trade  --intraday
```

```
#        --lots N  --max-loss Rs  --max-trades N
```

Output: <YYYY_MM_DD>\sensex_<date>.csv + nifty_<date>.csv (dated folder, auto-created) AND sensex_today.csv / nifty_today.csv (live mirror); paper_trades.csv when --trade. Needs today's upstox_token.json (run live_fetch.py first).

live_fetch.py [ACTIVE]

Purpose: Daily Upstox OAuth login - opens the browser, you paste back the ?code= URL, and it caches today's token in upstox_token.json. (It also has a legacy SQL-Server upsert of the day's close that the CSV pipeline ignores; the token is saved regardless.)

Run:

```
..\log_tradingVenv\Scripts\python.exe ..\code\live_fetch.py
..\log_tradingVenv\Scripts\python.exe ..\code\live_fetch.py --date 2026-06-16
```

Output: upstox_token.json (today's access token).

paper_tracker.py [ACTIVE]

Purpose: Forward DAILY carry-forward paper trader - the live strategy. LONG spread only, enter when daily z<=-2, hold across days until +70 / -100 pts / 30 days. Run once per day after the 15:30 close. Cannot over-trade (one position at a time).

Run:

```
..\log_tradingVenv\Scripts\python.exe ..\code\paper_tracker.py
# or feed the closes manually:
..\log_tradingVenv\Scripts\python.exe ..\code\paper_tracker.py --date 2026-06-25 \\
      --sensex-close 76900 --nifty-close 24010
```

Output: tracker_daily.csv (growing daily series), tracker_position.json (open trade), tracker_trades.csv (closed trades). Prints today's action.

live_trades_today.py [ACTIVE]

Purpose: Builds an Excel of TODAY's intraday trades computed straight from the live feed (works with or without the paper trader running). One-shot, or --watch rebuilds on every new trade until 15:30.

Run:

```
..\log_tradingVenv\Scripts\python.exe ..\code\live_trades_today.py
..\log_tradingVenv\Scripts\python.exe ..\code\live_trades_today.py --watch 10
# flags: --date  --sensex P  --nifty P  --watch [N]
```

Output: reports\live_trades_<date>.xlsx (Summary | Live Trades | Multiple Lots).

recover_data.py [ACTIVE]

Purpose: Rebuilds sensex_data.csv / nifty_data.csv from feed_run.log (price only) if the feed CSVs were lost. Edit the DATE constant at the top to the day you are recovering.

Run:

```
..\log_tradingVenv\Scripts\python.exe ..\code\recover_data.py
```

Output: sensex_data.csv / nifty_data.csv (tick_time, last_price).

old_data.py [ACTIVE]

Purpose: Downloads ~2 years of 1-minute OHLC from the Upstox historical-candle API into Old_data/YYYY/MM_Month/. Run it twice - once for NIFTY (default) and once for SENSEX. Needs today's token.

Run:

```
..\log_tradingVenv\Scripts\python.exe ..\code\old_data.py          # NIFTY
..\log_tradingVenv\Scripts\python.exe ..\code\old_data.py sensex   # SENSEX
```

Output: Old_data\YYYY\MM_Month\YYYY-MM-DD.csv (NIFTY) and sensex_YYYY-MM-DD.csv (SENSEX); columns timestamp,open,high,low,close,volume,open_interest.

backtest.py [ACTIVE]

Purpose: The strategy engine - compute_signals() + backtest() and all strategy parameters at the top (ENTRY 2 / EXIT 0.7 / STOP 100 / TARGET 30 / MAX_HOLD 15 / ratio 60 / lookback 15). Every report imports this so they all use the exact same logic. Not run on its own.

Run:

```
# imported by other scripts - not run directly
# change strategy params by editing the top of this file
```

Output: None (library). Edit params here to change the whole project's strategy.

backtest_intraday.py [ACTIVE]

Purpose: Applies the engine to one day's per-second ticks, with optional bar resampling and a cost model. Also provides load_series()/resample() reused by the report scripts.

Run:

```
..\log_tradingVenv\Scripts\python.exe ..\code\backtest_intraday.py \\
      --resample 1min --cost-pts 4 --out backtest_2026_06_25_1min.csv
..\log_tradingVenv\Scripts\python.exe ..\code\backtest_intraday.py   # per-second, raw
```

Output: A trade-log CSV at the --out path.

oldfolder_report.py [ACTIVE]

Purpose: Backtests the intraday strategy across ALL the 2-year 1-minute history in Old_data/ and writes two Excel reports (by month, by week) in the same format as your this_week_trade_detail file. RUN FROM code/ (not feed_data).

Run:

```
cd c:\Users\ADITYA\Desktop\Algo_trading\code
..\log_tradingVenv\Scripts\python.exe oldfolder_report.py
# or: oldfolder_report.py --root Old_data --out-dir ..\reports
```

Output: reports\old_data_monthwise.xlsx + reports\old_data_weekwise.xlsx.

cost_backtester.py [ACTIVE]

Purpose: Comprehensive cost-aware backtester: metrics, a parameter sweep, multi-lot and charts in one workbook. Costs are set to the Rs20/order basis (only Rs20 brokerage per order, other charges zeroed).

Run:

```
..\log_tradingVenv\Scripts\python.exe ..\code\cost_backtester.py \\
      --sensex sensex_today.csv --nifty nifty_today.csv --tag 2026_06_25
```

Output: One .xlsx: Summary, Trades 1min, Trades per-second, All Configs, Profitable Configs, Price Data, Multi-Lot.

edge_strategy.py [ACTIVE]

Purpose: Daily carry-forward research grid: LONG only, enters only when z<=-2 AND room-to-revert >= target, tests profit targets = 1x/1.5x/2x/3x the charge, and splits train (2024-25) vs test (2026). Reads history.csv (daily closes). Selective by design - does not over-trade.

Run:

```
..\log_tradingVenv\Scripts\python.exe ..\code\edge_strategy.py
# flags: --cost-pts 4   --split 2026-01-01   --out edge_strategy.xlsx
```

Output: edge_strategy.xlsx (Summary + Target-vs-Charge grid).

config.py [ACTIVE]

Purpose: Your Upstox API keys + sandbox token + (legacy) DB settings. Gitignored - holds secrets. Not run; edit it to configure credentials.

Run:

```
# edit values; never commit. Copy from config.example.py if missing.
```

Output: None (configuration).

config.example.py [ACTIVE]

Purpose: Blank template of config.py. Copy to config.py and fill in your keys.

Run:

```
copy ..\code\config.example.py ..\code\config.py
```

Output: None (template).

4. DEAD / ARCHIVED scripts (code/_archive/)

These are the original SQL-Server pipeline (need pyodbc + a SQL Server DB) or superseded backtest/report variants, kept for reference. They are NOT part of the current CSV workflow. To bring one back:

```
git mv code\_archive\<file>.py code\<file>.py
```

live_trader.py [LEGACY]

Purpose: Real-time tick-by-tick sandbox paper trader (old SQL version of tick_poller).

Run:

```
python live_trader.py [--source upstox] [--interval 2] [--orders]   # needs SQL + token
```

Output: SQL tables tick_data & paper_trade; live dashboard.

tick_feed.py [LEGACY]

Purpose: Real-time WebSocket feed from Upstox -> z-score signal, saved to SQL.

Run:

```
python tick_feed.py [--no-save]   # needs SQL + token
```

Output: SQL tick_data table; live dashboard.

raw_feed.py [LEGACY]

Purpose: Helper module: saves every tick with full market data to SQL raw_data (imported by live_trader).

Run:

```
# imported, not run directly
```

Output: SQL raw_data table.

algo_signal.py [LEGACY]

Purpose: Computes the live spread z-score and prints the current LONG/SHORT/HOLD signal.

Run:

```
python algo_signal.py [--source upstox] [--no-live]   # needs SQL
```

Output: Prints signal (no file).

check_trade.py [LEGACY]

Purpose: Live paper-trade dashboard: spread, z, open P&L, session totals.

Run:

```
python check_trade.py   # needs SQL + token
```

Output: Prints dashboard (no file).

live_fetch.py (SQL parts) [LEGACY]

Purpose: (See active live_fetch.py) - the SQL upsert of the day's close is legacy.

Run:

```
# token still works; SQL upsert needs pyodbc + DB
```

Output: upstox_token.json (+ SQL price_data).

fill_real_data.py [LEGACY]

Purpose: Downloads SENSEX/NIFTY history from Yahoo Finance into SQL yfinance_feed.

Run:

```
python fill_real_data.py [--start 2025-01-01] [--end 2025-12-31] [--yes]   # needs SQL
```

Output: Populates SQL yfinance_feed.

generate_data.py [LEGACY]

Purpose: Generates synthetic cointegrated SENSEX/NIFTY series for testing.

Run:

```
python generate_data.py
```

Output: price_data.csv (750 daily bars).

export_sql.py [LEGACY]

Purpose: Turns backtest CSVs into a standalone .sql file (CREATE TABLE + INSERT).

Run:

```
python export_sql.py   # reads trades.csv, price_data.csv
```

Output: ratio_strategy.sql.

optimize.py [LEGACY]

Purpose: Grid-search + walk-forward (train 2023-24, test 2025-26) over 1296 combos.

Run:

```
python optimize.py [--source upstox] [--apply]   # needs SQL
```

Output: optimization_results.csv; prints top 15. --apply writes best params into backtest.py.

report.py [LEGACY]

Purpose: 4-sheet formatted Excel backtest report from SQL data.

Run:

```
python report.py [--start 2023-01-01] [--end 2025-12-31] [--out X.xlsx]   # needs SQL
```

Output: backtest_report_<date>.xlsx.

backtest_daily.py [LEGACY]

Purpose: Multi-day position-holding backtest on daily data.

Run:

```
python ..\code\backtest_daily.py --cost-pts 4 [--target 200] [--maxhold 25]
```

Output: backtest_daily_report.xlsx + two trade CSVs.

backtest_june.py [LEGACY]

Purpose: Single-month (default June 2026) multi-day backtest held to monthly expiry.

Run:

```
python ..\code\backtest_june.py [--month 2026-06] --cost-pts 4 [--target 200]
```

Output: backtest_2026_06_monthly.xlsx.

bigmove_strategy.py [LEGACY]

Purpose: Daily strategy that only books big targets (50/70/100 pts); train vs test.

Run:

```
python ..\code\bigmove_strategy.py --cost-pts 4 --split 2026-01-01
```

Output: bigmove_strategy.xlsx.

profit_strategy.py [LEGACY]

Purpose: Daily LONG strategy, winners run to reversion; out-of-sample split.

Run:

```
python ..\code\profit_strategy.py --cost-pts 4 --split 2026-01-01
```

Output: profit_strategy.xlsx (4 sheets).

profit_strategy_filtered.py [LEGACY]

Purpose: Adds a trend filter to profit_strategy; compares filter ON vs OFF.

Run:

```
python ..\code\profit_strategy_filtered.py --cost-pts 4 --split 2026-01-01
```

Output: profit_strategy_filtered.xlsx.

optimize_intraday.py [LEGACY]

Purpose: Parameter sweep on one day of tick data (warns: in-sample curve-fitting).

Run:

```
python ..\code\optimize_intraday.py --cost-pts 4 --tag 2026_06_25
```

Output: all_configs.csv, configs.csv, best.csv (for the tag).

walkforward.py [LEGACY]

Purpose: Out-of-sample AM-train / PM-test on tick data.

Run:

```
python ..\code\walkforward.py [--folds 2] --cost-pts 4
```

Output: walkforward_<date>.csv.

multiday_report.py [LEGACY]

Purpose: Aggregates several feed days + cross-day walk-forward (DAYS list hardcoded).

Run:

```
python ..\code\multiday_report.py
```

Output: multiday_report.xlsx + 3 CSVs.

make_summary.py [LEGACY]

Purpose: Plain-language verdict CSV from backtest outputs for a tag.

Run:

```
python ..\code\make_summary.py --tag 2026_06_25
```

Output: backtest_<tag>_SUMMARY.csv.

build_excel_report.py [LEGACY]

Purpose: Consolidates a day's backtest CSVs into one 6-sheet workbook.

Run:

```
python ..\code\build_excel_report.py --tag 2026_06_25
```

Output: backtest_<tag>_report.xlsx.

build_onepager.py [LEGACY]

Purpose: One-page Word briefing of a day's results from the SUMMARY csv.

Run:

```
python ..\code\build_onepager.py --tag 2026_06_25
```

Output: backtest_<tag>_onepager.docx.

feed_to_excel.py [LEGACY]

Purpose: Dumps the per-second tick CSVs into one 2-sheet Excel workbook.

Run:

```
python ..\code\feed_to_excel.py --tag 2026_06_25
```

Output: feed_data_<tag>.xlsx.

paper_trades_excel.py [LEGACY]

Purpose: Excel P&L report from paper_trades.csv with rupee conversions.

Run:

```
python ..\code\paper_trades_excel.py --rs-per-point 20
```

Output: paper_trades_pnl.xlsx (3 sheets).

5. Folder layout

code/ - all scripts + config.py (active set) ; code/_archive/ = retired scripts
feed_data/ - tick CSVs + runtime files (upstox_token.json, history.csv, instruments.json.gz, tracker_*.csv) ; dated subfolders like 2026_06_25/ hold a day's archived feed
code/Old_data/ - 2 years of 1-minute history (YYYY/MM_Month/) for the big backtest
reports/ - all generated XLSX / DOCX / PDF outputs

Tip: data CSVs cannot be re-fetched once a market day closes - back up the whole folder (zip) regularly.